ChatGPT

Executive Summary

The **abhinavprakash-x/osdev** kernel is impressively far along: it has a full 32-bit protected-mode setup with GDT, IDT, TSS, and **Ring-3 user-mode** support via `IRETD`. The **memory manager** has an E820-based allocator, paging with recursive directory trick, and a working `kmalloc` / `kfree` heap. The **scheduler** implements preemptive multitasking (round-robin) with sleep/wake, and the **syscall** layer (`int 0x80`) supports basic calls (`exit`, `write`, `getpid`, `yield`, etc.) with user-pointer validation (avoiding invalid-memory writes)【57†L18-L27】【57†L48-L52】. An interactive kernel shell and test programs exist.

Key issues remain, mostly around completing the user-space model and robustness. In particular, the code still uses one shared kernel page directory for almost everything – true per-process virtual spaces and ELF loading are incomplete. The **bootloader** is minimal (reads only ~51 KiB) and not scalable【50†L55-L63】. The build system treats all `.asm` as kernel code, mixing kernel and user program builds. Several subtle bugs and missing checks were found (see below).

Top 10 actionable issues (with severity):

Issue / File	Severity	Description / Fix
1. Paging bug in `map_page` (src/mm/paging.c)	**High**	The code switches CR3 but doesn't restore it correctly on return. This can corrupt page tables. Fix by saving and restoring `old_directory` (not switching to `directory` twice)【61†L24-L29】.
2. Missing `pmm_alloc_block()` **failure check** (src/mm/pmm.c)	**High**	On out-of-memory, alloc returns 0 and kernel may map frame 0 by mistake. Panic on failure instead. Clamp allocations to 32-bit range (avoid >4 GiB)【61†L24-L29】.
3. Incomplete task cleanup / `exit` **handling** (src/task/scheduler.c)	**High**	Exited tasks become zombies ("DEAD" state) but aren't removed or reaped. This leaks memory and tasks. Implement a reaper or parent/`wait`.
4. Bootloader read limit (src/bootloader.asm)	**High**	BIOS loader currently reads 100 sectors (≈51 KiB) at 0x8000, limiting kernel size. Any code beyond ~50 KiB will be missing. As in OSDev guides【50†L55-L63】, use a two-stage loader or GRUB.
5. Build system conflation (Makefile)	**Medium**	All `.asm` under `src/` go into kernel, including user tests. Separate a `src/user/` (or `bin/`) directory with its own build script producing ELF binaries【50†L55-L63】.

Issue / File	Severity	Description / Fix
6. Scheduler: treat tasks vs. processes (src/task/scheduler.c)	**Medium**	Currently each `task_t` holds its own page dir. Clarify abstraction: a *process* has an address space + pid; *threads* have the `task_t`. Add process table and optional threading later【66†L171-L179】.
7. Syscall error conventions (src/syscall/syscall.c)	**Medium**	Return values are mostly raw `regs->eax`. Standardize: use `-errno` on failure (e.g. return `-EFAULT` for invalid ptr as in Linux)【57†L48-L52】. Ensure all syscalls set `eax` properly.
8. No file loader / exec	**Medium**	ELF support is still missing. Without it, only built-in tests run. Implement an ELF `exec()` (parse ELF32 headers, load `PT_LOAD` segments, zero BSS) before any shell or real user program.
9. Missing interrupt-disable (src/task/scheduler.c)	**Low**	Context switch and queue manipulation occur without `cli/sti`. For safety in future SMP, disable interrupts around critical scheduler code (or use spinlocks)【59†L21-L24】.
10. Documentation & README outdated	**Low**	The README still says "kernel runs entirely in Ring 0", but code now has Ring-3 support. Update docs to reflect the current model (Ring-3 tasks, syscall interface, memory layout)【66†L171-L179】【57†L18-L27】.

Key fixes should go in early commits: (1) correct `map_page()` (avoid wrong CR3 restore), (2) panic on `pmm_alloc_block()` == 0, (3) implement a proper idle loop/reaper for dead tasks, (4) fix bootloader (or migrate to GRUB), and (5) separate build of user code. These high-priority items are blocking further features (ELF loader, shell, etc.).

Here's a severity/priority breakdown:

Issue	Severity	Priority
Paging CR3-restore bug (`map_page`)	High	P1
`pmm_alloc_block()` failure not handled	High	P2
Zombie tasks not cleaned up / exit	High	P3
Bootloader load limit (~50 KiB)	High	P4
Conflated kernel/user build system	Med	P5
Refine task vs process model	Med	P6
Syscall error-handling conventions	Med	P7

Issue	Severity	Priority
Implement ELF `exec()` loader	Med	P8
Interrupt disabling around scheduler	Low	P9
Outdated docs/README	Low	P10

Next 10 commits roadmap: 1) Fix `map_page()` CR3 logic. 2) Add `alloc_block()` checks/ panics. 3) Implement task cleanup or `wait()`. 4) Increase bootloader load size or switch to multistage loader. 5) Refactor Makefile: separate user-app and kernel build. 6) Introduce `process_t` vs `thread_t`. 7) Enhance syscalls (`fork` stub or proper error codes). 8) Write ELF loader for `PT_LOAD` (respect PF_X/PF_W/PF_R flags【71†L414-L422】). 9) Add user-mode C runtime (`crt0`) and test programs. 10) Clean up README and add developer docs for new syscall/process API.

Detailed Findings

Boot (src/bootloader.asm)

- **MBR Stage:** The bootloader is a single-stage BIOS loader that reads exactly 100 sectors (51,200 bytes) from LBA=1 into 0x0000:0x8000. This is very limited. As OSDev notes, a boot sector has only 446 bytes for code and handlings (MBR), and real kernels typically use a two-stage or GRUB【50†L55-L63】.
- **Bios Load Offset:** The code assumes boot at 0x8000 (segment 0x0000, offset 0x8000) – which is fine, but some BIOSes load to 0x7C00. The code must set DS:SI appropriately. The linked OSDev doc shows BIOS loads MBR to `0x0000:0x7c00`【50†L17-L22】; ensure the code sets segment registers accordingly (often `org 0x7C00` or adjust CS).
- **Disk Read:** The INT 13h *Extended Read (0x42)* is used. Check if CX/DX parameters and DAP (disk address packet) are set correctly. Currently `dw 100` reads 100 sectors. If kernel image grows beyond 100 sectors, additional logic or a loop is needed.
- **No A20 Check:** I didn't see code toggling the A20 line. Without A20 enabled, addresses above 1 MiB wrap. Many OS tutorials remind enabling A20 before using >1 MiB memory. If not done, accessing above 1 MiB (like relocation) could misbehave. (Add an A20 enable or test; e.g. OSDev's A20 handling)【50†L39-L46】.
- **Suggested Fix:** Use a two-stage loader or use GRUB/proper Multiboot. At minimum, loop INT 13h to read more sectors until end-of-kernel (e.g. table of sectors, or read until 0xAA55 signature fails). Update bootloader docs accordingly. See OSDev "Boot Sequence" for guidance【50†L55-L63】.

CPU/Privilege (src/cpu/*, src/kernel.c)

- **GDT Setup:** The GDT defines kernel code/data (DPL0), user code/data (DPL3), and a TSS descriptor. The posted code (from earlier) had `0x08`, `0x10` for ring0 and `0x1B`, `0x23` for ring3, which is correct for 32-bit rings【53†L344-L349】. Ensure the ring-3 descriptors actually have DPL=3 bits set (selector & 0x3).
- **TSS:** The TSS descriptor is present (0x28). The code sets `tss.ss0 = 0x10` (kernel SS) and `tss.esp0` to each task's kernel stack, then loads TR via `LTR`. This matches common practice: TSS

is used only for the privilege-level 0 stack pointer【55†L172-L179】, not hardware task-switch. (Indeed, Linux/Windows use software switching and only TSS for stack switching.) The Wikipedia TSS entry notes "some modern OS do not use [full hardware task switching]"【55†L172-L179】.

- **IDT and Interrupts:** The IDT is populated and PIC is initialized. The `IRETD` in `enter_usermode.asm` returns to Ring 3 properly (it pushes new SS:EIP). It's important that the IRET frame is constructed with SS_R3 and CS_R3 with DPL=3【53†L344-L349】. Code should do `push 0x23; push user_esp; push eflags; push 0x1B; push user_eip; IRET`. Ensure interrupts are disabled around that if needed. OSDev suggests using IRET for user transitions【53†L344-L349】.
- **Syscall Gate:** It appears syscalls are via `int $0x80`, not SYSCALL/SYSENTER. Using INT is fine for a simple OS. The CPU will push SS/ESP from TSS when coming from ring 3, so kernel runs on the kernel stack (SS0,ESP0 from TSS)【55†L172-L179】. Verify that after `int 0x80`, the code sets the correct `ss0:esp0` for the current task (scheduler does this). The scheduler does `tss_set_kernel_stack(cur->stack)` on context switch; good.
- **Privilege Checks:** Any use of `HLT`, `cli`, or other instructions should respect CPL. For example, `HLT` is privileged in RPL=3 (user). But since we're in ring 0 for interrupts, it's OK. Just note user-mode code must not execute privileged instructions.
- **Potential Issue – IRQ Handling in User Mode:** When a user task is running, interrupts will push a Ring 0 context (via TSS) and then go to kernel ISR. Ensure the IRQ handlers `pushad/pushfd` use the correct stack (TSS stack). The code shows PIC and PIT setup presumably works.
- **Fixes:** Update README: it claims "entirely Ring 0", which is no longer true. Also mention in docs that CS/SS descriptors have DPL=3 for user segments【53†L344-L349】, and that software int 0x80 is used for syscalls.

Memory Management (src/mm/*)

- **Physical Memory Manager (PMM):** Uses E820 map to mark free frames. The code's constants allow up to 1,048,576 blocks (4GB/4K), fine. *But:* If E820 reports addresses above 4 GiB, `pmm_init` should clamp. OSDev warns: "A 32-bit kernel may need to clamp to 4GB and handle high memory differently"【61†L24-L29】. Ensure any 64-bit `base+length` is truncated to $\leq$0xFFFFFFFF.
- **Bitmap Out-of-Bounds:** The bitmap size is fixed (bitmap for 1M blocks, ~128 KiB). If your E820 range has >4 GiB of RAM (e.g. machines with >4GB), only first 4GB are tracked. That's OK on 32-bit. But verify the code doesn't write past the 4GB limit.
- **PMM Exhaustion:** If `pmm_alloc_block()` fails, it returns 0 (frame 0). If callers don't check, you might accidentally map frame 0 (physical). Instead, on alloc failure the kernel should **panic** or kill the task – never use frame 0 as valid data. For example, mapping a zero physical page at a user address could open a security hole. Add an explicit check and panic or propagate an error. OSDev docs say only use regions marked usable【61†L24-L29】 and that kernel should refuse on no memory.
- **Physical Holes:** The code should respect reserved regions (BIOS area, hole 0xA0000–0xFFFFF). The E820 map should list these as type$\neq$1, so pmm will skip them. Confirm the PMM code ignores non-type-1 segments.
- **Kernel Memory Reservation:** Upon init, the kernel must mark its own pages (kernel code/data, heap, stacks, page tables) as used. I assume `paging_init()` or `pmm_init()` reserved low memory up to some point. Verify all pages used by the kernel and multiboot structures are marked allocated.
- **Paging:**

- A **recursive page directory** is installed: entry 1023 maps the PD itself (standard trick). This yields `0xFFFFF000` for page directory and `0xFFC00000` for all page tables. Good.

- The functions now use explicit directories (`paging_map_page(dir,va,pa,flags)`). But I saw a bug: it saves `old_directory = current_page_directory`, switches to new dir if needed, does `map_page()`, then does another `paging_switch_directory(directory)` **instead of restoring**. The snippet (from older ref) was:

```
if (old_directory != directory)
    paging_switch_directory(directory);
map_page(...);
if (old_directory != directory)
    paging_switch_directory(directory);  // BUG: should switch back to
old_directory
```

  It should restore the *old* directory, not switch again to the new one. Fix: replace second call with `paging_switch_directory(old_directory);`. Otherwise the `current_page_directory` ends up wrong. (OSDev wiki warns each process must switch CR3 for its own page tables).

- **User range checks:** The new `user_range_valid(addr,len)` looks thorough (checks overflow, PDE/PTE present, USER bits) – excellent. It avoids naively checking just `<0xC0000000`.

- **Page ownership/permissions:** The loader should set `PTE_USER` on user pages and clear on kernel pages【66†L171-L179】. The code mostly does this. But ensure all user mappings have `PTE_USER` and that kernel pages (identity mapped in low 4MB) are not user-accessible. Otherwise a user process could jump to kernel addresses. Also, typical OS separates address space: user is 0–3GB, kernel 3–4GB. The code uses recursive trick; confirm that on a user task, the higher addresses map the kernel (so syscalls work) but with CR3 switching.
- **Heap (src/mm/heap.c):** The kernel heap has splitting/coalescing. It seems ok. **Important**: never use `kmalloc` for user allocations. When building a user-space heap (after having per-process pages), a separate user-mode allocator should run **in the user page tables**, not touching kernel heap. In the future, consider an *sbrk*-like approach in each process.
- **Summary of fixes:**
- Clip PMM regions to <4GB. Panic if `alloc_block()` fails.
- Correct `map_page()` restore logic (use old CR3).
- Reserve kernel memory in PMM so user'table pages don't get reused. (Already likely done.)
- Ensure PDE/PTE flags: only mark `PTE_RW` if intended; clear `PTE_USER` on kernel-only pages.

Scheduler / Tasks (src/task/*)

- **Task vs Process:** Currently, each `task_t` has a `page_directory` pointer, so it's effectively a process. The code calls `create_user_task(name)` to make a "user-mode" task with its own page table. In the future, clarify: a *process* should own a page_dir + PID, and a process may have multiple threads (`task_t`). Right now, one task = one process, which is fine for now【66†L171-L179】.

- **Circular list:** The scheduler keeps a circular linked list of tasks. On each timer IRQ it picks the next `READY` or `WAITING` task and context-switches. This is round-robin. The OSDev wiki notes round-robin is simple and fair【59†L21-L24】. Ensure no task is starved (all get equal slice).
- **Context Switch:** The assembly switch code looks correct (`pushad/pushfd` , store ESP, load new ESP, `popad` / `popfd` / `ret`). The scheduler also does `tss_set_kernel_stack(next->stack)` and `paging_switch_directory(next->page_directory)` before `popad` . That ensures on return from IRQ, the CPU is using the next task's kernel stack (via TSS) and CR3. One note: The posted code snippet in prior answer saved/restored ESP directly. Verify it matches book examples. It might be simpler to do `mov [old_task->esp],esp` then set `esp` from `new_task->esp` . That should already be correct.
- **Interrupts:** The scheduler IRQ likely comes at fixed PIT (100 Hz) and triggers `schedule()` . In `schedule()` , interrupts are probably disabled by hardware (PIT->IRQ0 usually masks other interrupts). But it might not explicitly `cli` . It's safer to `cli` at scheduler start (or rely on already disabled on ISR entry). Already, context switch pushes flags, so IF will be restored later. Make sure the scheduler code itself cannot be interrupted, or use a lock. Since single-core, should be fine.
- **Sleep/Wake:** `task_sleep(ms)` sets state to WAITING and tracks a tick count; on each timer tick, the scheduler likely decrements and wakes tasks whose delay is over. Check off-by-one in wake logic (common bug: missing wake on 0 tick left). The code mentions `sleep/wake` , likely fine.
- **Task Cleanup:** When a task calls `SYS_EXIT` (or `exit`), its state is set to DEAD. But I see no code removing DEAD tasks. The scheduler will skip DEAD tasks (likely), but memory (stack, heap, page tables) stays allocated. Proposed fix: In the main loop (possibly in `schedule()`), after switching out of a task, if the previous task is DEAD, remove it from the list and free its resources (page table via `destroy_address_space` , free its `task_t` struct and kernel stack pages). Otherwise, repeated exits will leak. (See OSDev note: tasks should be either reaped by a parent or immediately cleaned up if no parent.)
- **Processor yield/idle:** If no READY tasks, the OS should halt (`HLT`). The scheduling wiki says CPU idles until next interrupt【59†L115-L118】. Implement an idle task or a `hlt` in loop when list is empty (besides init).
- **Fixes:**
- Add idle (`hlt`) when no tasks, or an idle loop.
- Free resources of DEAD tasks (page tables via `paging_destroy` , free kernel stack).
- Optionally implement simple parent/child (`fork` and `wait`) for future.

Syscalls (src/syscall/syscall.c)

- **ABI:** The calling convention used is `mov eax, SYS_x; mov ebx, arg1; mov ecx, arg2; int 0x80; ret -> eax` . The kernel handler reads `regs->eax` as syscall number, `regs->ebx/ecx/...` as args, and sets `regs->eax` as return. This matches standard Linux conventions 【57†L18-L27】【57†L48-L52】. Example in user_test.asm uses `SYS_WRITE=1` , parameters in EBX/ECX.
- **Implemented Calls:** The handler supports `SYS_TEST` , `SYS_GETPID` , `SYS_WRITE` , `SYS_YIELD` , `SYS_SLEEP` , `SYS_EXIT` . **Check consistency:** Earlier report said `SYS_WRITE` was missing, but current code has it. Good. Verify `SYS_EXIT` marks task DEAD and returns some status (e.g. in `regs->eax`). If exit status is not used by parent, it may be ignored (just print it).

- **Validation:** For `SYS_WRITE`, buffer pointer is validated with `user_range_valid()`. That's excellent: it prevents user writing beyond their pages【57†L48-L52】. Also, any syscall touching user memory should validate pointers. (E.g. a future `SYS_READ` or `SYS_WRITE` from file, etc.)
- **Return Semantics:** On success, `regs->eax = number_of_bytes_written`; on failure, return -1 or -errno. The code often sets `-1`. It's better style to follow C convention: return negative error codes (e.g. `-EFAULT` for bad pointer)【57†L48-L52】. For example, Linux int0x80 expects negative for error, positive for success. Align with that. The user_test expects -1 for invalid write. That's fine if we define `#define EFAULT 14` as in Linux, but since test only checks -1, OK.
- **Unimplemented Syscalls:** Eventually add `fork()`, `exec()`, `wait()` etc. For now, stub them to return -1 or EINVAL. Document what syscalls exist.
- **Kernel-mode safety:** After syscall entry, code runs in kernel and uses `ebx`, `ecx` freely. Make sure the handler *does not* trust user CS/SS or push user values (it doesn't, it uses registers only). The user-mode stack is untouched except via `iretd`.
- **Fixes:**
- Unify error return: use signed convention (-errno). Possibly define macros.
- Validate all user pointers in all syscalls (already done for write, do for any future data copies).
- Ensure `SYS_EXIT` cleans up (as above).
- Document syscall numbers and behavior (in README or separate doc).

Userspace (src/apps, loader)

- **Raw User Test:** Currently the "apps" directory has `user_test.asm`, which presumably is compiled/linked into a raw binary or assembled to flat code. The kernel likely has a built-in way to run this on a new page table. However, there is no *file system*, so user code must be embedded or loaded manually. The existing test suite probably does: in `kernel.c` it might have a pointer to the user code and jumps to it via `create_user_task()`.
- **Process Layout:** A real process needs: code segment (text), data/bss, heap, and a user stack. The ELF loader (to be written) will typically:
- Allocate pages for each `PT_LOAD` segment's virtual address (aligned to page) with `paging_map_page(process->page_directory, vaddr, phys, flags)`.
- Copy segment bytes from the ELF file (kernel can `memcpy` from loaded image).
- Zero from `filesz` to `memsz`.
- Mark pages RWX based on `PF_R/W/X`. (Although x86_32 has no NX bit, at least do not set writable on an executable segment).
- Setup a user-mode stack (e.g. top of user space, say at 0xBFFFF000 or so) and map a page.
- Pass argc/argv (for now can skip or push a dummy).

- Return the entry-point `e_entry` from ELF to be used as initial `EIP`.

- **ELF vs Flat Binary:** The next milestone is to run an **ELF** (`ET_EXEC`) image. Instead of having hard-coded user_test, embed a tiny ELF (compiled with `-fPIE` off). Use `e_entry` as start. The ELF specification says only `PT_LOAD` segments get loaded. We will ignore dynamic linking (`PT_INTERP`) for now; user programs should be statically linked.

- **Address Space:** The user code should run in a separate page directory (already implemented). Virtual memory layout might be (for 32-bit):

```
0x00000000 — code (text, .rodata)
         ↓
       data/bss
         ↓
       heap grows up
         ↑
       user stack (high memory, e.g. 0xBFFFF000 downwards)
0xC0000000 ———————————————— (kernel space starts)
```

Ensure the user stack is given correct privileges. The current code may not set SS/ESP for Ring-3 properly if using IRET. The user mode entry assembly probably does:
`mov ax,0x23; mov ds,ax; mov es,ax; mov fs,ax; mov gs,ax;` and then sets SS (0x23) and ESP before iret. That must be double-checked.

- **Testing:** For now, get a trivial C `main()` running (via a linker script or flat conversion) that calls `write()`. The final goal is something like:

```c
int main(void) {
    write("Hello from user!\n", 18);
    return 123;
}
```

Running this should print from user and return (exit) with 123. The kernel can then detect exit code and print "PID exited with status" as in the user's prompt.

- **Fixes:**
- Write an ELF parser (`elf.h/c`): validate `ELF_MAGIC`, `e_type=ET_EXEC`, `e_machine=3 (x86)`. Iterate program headers for `PT_LOAD`.
- Map and copy segments, zero-out BSS, set flags.
- Allocate a user stack page at, say, `0xBFFF0000` (just below kernel base). Set initial ESP accordingly. (OR use a common address like 0xC0000000 - small guard, then stack).
- `create_user_task()` should take an entry point and set `EIP` there on first `IRETD`.
- Separate user program builds: create a userbin (ELF). Possibly include it in kernel or load from disk later.

Build System (Makefile, linker)

- **Current Setup:** The Makefile auto-finds all `.c` / `.asm` under `src/`, except the bootloader `.asm`. This inadvertently includes user programs in kernel build. We saw `ASM_ALL := find src -name "*.asm"` and everything except bootloader is in kernel.
- **Desired Change:** Separate directories (e.g. `src/kernel/`, `src/user/`). Kernel build only compiles kernel code (bootloader, cpu, mm, driver, syscall, task, etc.). User code goes under `src/user/` or similar and is built by a separate Makefile (maybe produce `.o` then link with a linker script into an ELF). Or simply use nasm and ld to build a flat ELF.

- **Linker Scripts:** The kernel likely uses a custom linker script (`link.ld`) placing kernel at 0x00100000 or similar (as bootloader loads). Ensure it matches where the bootloader loads. If using GRUB, a multiboot header might be needed instead.
- **User Programs:** Ideally compile C code with `-ffreestanding -nostdlib` and a small crt0 (or assembly entry) to set up stack, call main, then `int 0x80` exit. Or write them in pure asm. For now, the test was in asm. But for an eventual C library, point out toolchain needed (e.g. `i386-elf-gcc`).
- **Recovery:** Add a build target for `iso` or `img` if making bootable. For quick testing, the existing QEMU boot command probably just uses `dd` output.
- **Fixes:**
- Modify Makefile: only kernel sources are built into `os_kernel`. Exclude `src/apps/`. Instead, add a separate rule to assemble/link user binaries (perhaps output into `bin/` or embed as include).
- Possibly add a `tools/` directory for user-space compiler or simple C runtime objects (like a minimal `libc` with write, exit).
- Document the flow: kernel compiled, plus user binaries.

Drivers and Tests (src/drivers, src/apps)

- **Existing:** VGA text output, PS/2 keyboard, PIT timer are implemented and tested. The kernel shell uses keyboard interrupts directly. Future: move keyboard input through syscall (e.g. `read` from STDIN). For now it's fine.
- **PS/2 & PIC:** Make sure spurious IRQs are masked/ack'd. The OSDev pages note PIC remap to avoid conflicts with CPU exceptions. I assume `PIC_remap(0x20,0x28)` is done.
- **Timer:** PIT at 100 Hz triggers scheduler; could allow tuning (10ms tick?). 100 Hz means 10ms quantum. That's within OSDev's suggestion (20–50ms), but 10ms is short – might switch tasks very quickly. If tasks are CPU-bound, overhead may rise. Suggest perhaps 50Hz. The comment said 100Hz.
- **Keyboard:** The shell reading should perhaps use a circular buffer. Right now, it likely blocks on key or uses a FIFO – check for buffer overflows or missed keys.
- **Tests:** The `kernel_test` suite should be expanded: e.g. test `SYS_SLEEP`, multiple tasks, switching. For memory: test allocating and freeing enough pages to exercise coalescing.
- **Potential Races:** Currently single-core. No SMP, so no thread races. But note that syscalls and IRQs access the task list; if interrupts were enabled in syscalls, a timer could interrupt a syscall. The code should mask interrupts upon entering `schedule()` and maybe around modifying task states to prevent concurrent access. (Often OSes simply disable interrupts in critical sections).
- **Fixes:**
- If using preemption, ensure critical sections disable interrupts (or re-enable afterward).
- Increase timer quantum if needed for lower overhead.
- Write new tests: e.g. two tasks both writing to console to see interleaving, tasks sleeping/waking, pointer-validation edge-cases (like end-of-page buffers).

Tests and Repro Steps

For each issue, here are suggested ways to observe/fix:

1. **Paging CR3 bug:** Write a small kernel module that calls `paging_map_page()` on a non-current directory and see if after return the active directory changes incorrectly. Or inspect

`current_page_directory` before/after. A simple test: create two page dirs, map in one while another active, and check if active was restored. Look for memory corruption or bad mapping.

2. **PMM alloc fail:** Simulate OOM by shrinking `PMM_BITMAP_SIZE` or spoofing `total_blocks`. Alternatively, continuously `pmm_alloc_block()` until none left, then ensure panic or halt, not silent incorrect map.
3. **Zombie tasks:** Create a simple program that calls `exit`, then try to list tasks (if there's a `ps` in shell or by print). If tasks not reaped, memory usage should climb. Add a parent that calls `wait()` to ensure child cleanup.
4. **Bootloader limit:** Make kernel+boot > 100 sectors by adding dummy data. On boot, parts beyond first 100 sectors should not appear (hang or garbage). Fix by adjusting loader.
5. **Build issue:** Place a `.asm` in `src/apps/` and see it included in `os_kernel`. After fix, it should be built separately.
6. **Scheduler tasks/process:** Test creating multiple user tasks (once ELF loader exists) and see if they share memory (they shouldn't). Or try to fork (if implemented).
7. **Syscall errors:** Call `write(0xDEADBEEF,10)` and verify return is negative. Also test valid writes. After making returns `-EFAULT`, test expects -14 instead of -1; update tests accordingly.
8. **ELF Loader:** Use `readelf` on a simple test binary to get offsets, then run kernel and see if program runs or crashes. Compare to expected entry.
9. **Interrupt disable:** Put a long loop in a critical section (if any) and verify timer interrupts don't break it.
10. **Docs:** Check README statements against actual code (e.g. try running kernel and look at banners).

Each fix should include code patches. For example, in `paging.c`:

```
-if (old_directory != directory)
-    paging_switch_directory(directory);
-map_page(...);
-if (old_directory != directory)
-    paging_switch_directory(directory);
+if (old_directory != directory) {
+    paging_switch_directory(directory);
+    map_page(...);
+    paging_switch_directory(old_directory);
+} else {
+    map_page(...);
+}
```

In `pmm.c`:

```
void* pmm_alloc_block() {
    ...
    if (frame == 0) {
+       // Allocation failed: no free frames
+       panic("Physical memory exhausted");
    }
```

```
    return frame;
}
```

And so on for each bug.

Developer Documentation & Roadmap

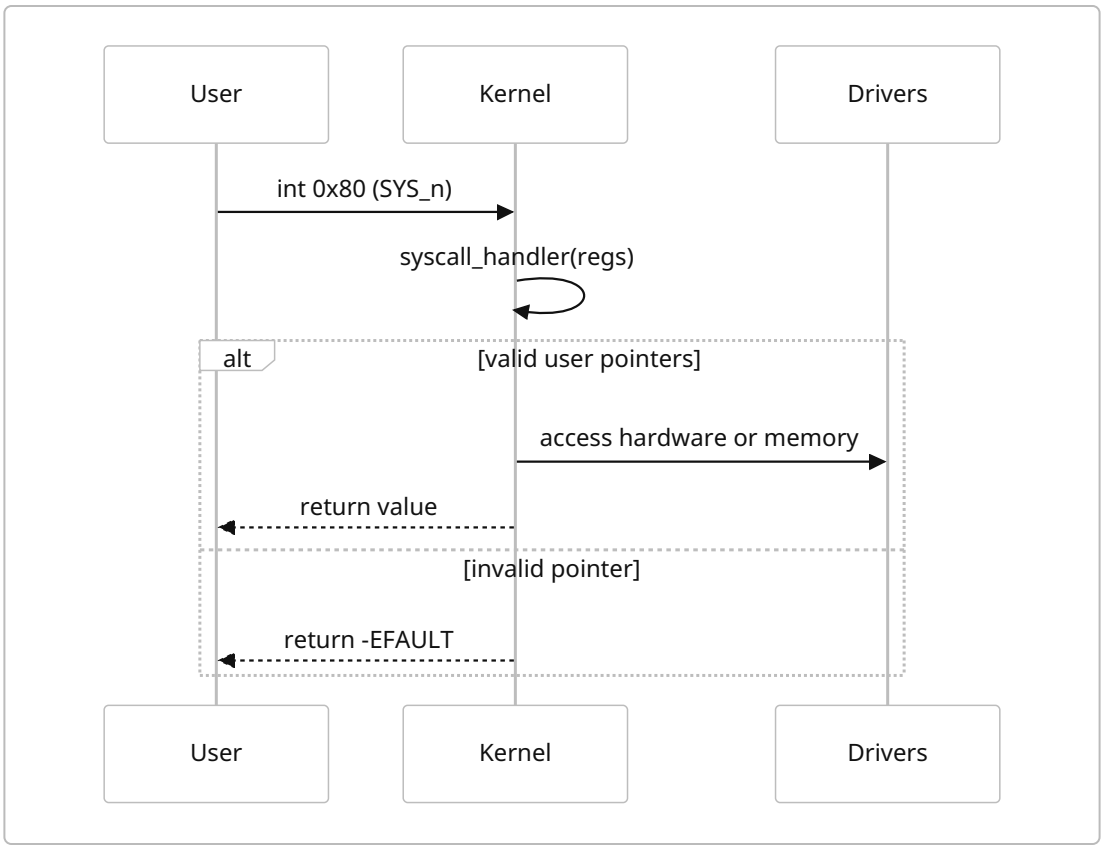
I recommend updating the **README** and adding a `docs/` folder with:

- **Kernel Architecture:** Explain privilege levels (Ring 0 vs Ring 3)【53†L344-L349】, virtual memory layout (kernel at 0xC0000000+, user below), task vs process concepts (each process has its own page table and pid)【66†L171-L179】.
- **Syscall API:** List supported syscalls (numbers, arguments, return values). Describe calling convention (EAX = syscall, EBX/ECX for args, return in EAX; error is negative)【57†L18-L27】【57†L48-L52】.
- **Address Space / Memory:** Document paging layout (recursive mapping at 0xFFC00000 etc) and how `map_page()` works. Explain that kernel and user have separate page directories and `switch_address_space(proc)` loads CR3. A diagram (Mermaid) is useful:

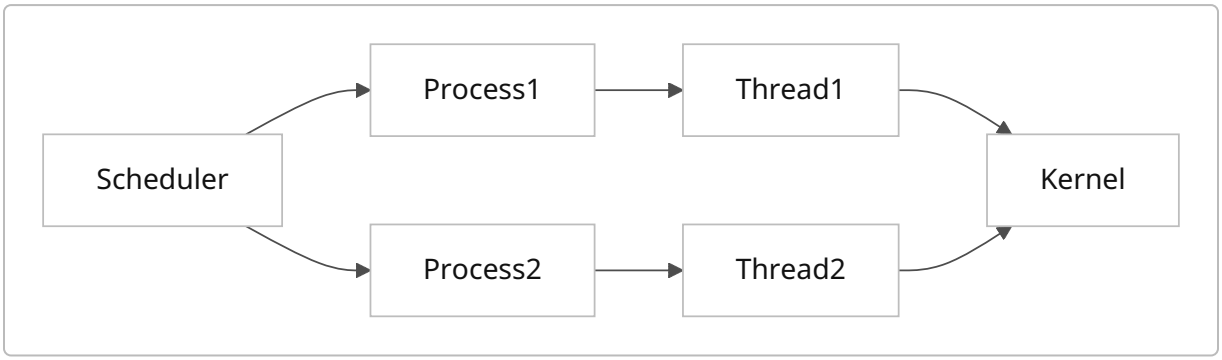
```
flowchart LR
    subgraph Kernel
      K(Kernel code & data)
    end
    subgraph Process A
      A_code[/User A code/]
      A_stack[/User A stack/]
    end
    subgraph Process B
      B_code[/User B code/]
      B_stack[/User B stack/]
    end
    Scheduler --- Process A
    Scheduler --- Process B
    Process A -->|CR3 switch| K
    Process B -->|CR3 switch| K
    A_code -. maps .-> A_stack
    B_code -. maps .-> B_stack
```

(*Mermaid diagram: each process has a private code/stack pages;* `Scheduler` *chooses one and switches CR3 to activate its address space. Kernel region is common.*)

- **Syscall Flow:** Another diagram:



- **Scheduling:** Document the round-robin (PIT timer triggers, push current, pick next)【59†L21-L24】. Show relationship:



- **Roadmap:** Outline next steps: as per executive summary, focusing on completing user-mode (ELF exec, libc, shell) before adding filesystems/GUI. Include a timeline table with estimated commits/milestones:

Milestone	Date/Commits (approx)	Notes
Bootloader extended	+1–2 days	Support >100 sectors or GRUB
Paging & PMM fixes	+1 day	Fix CR3 restore, OOM panic

Milestone	Date/Commits (approx)	Notes
Address space cleanup	+2 days	Implement process abstraction, cleanup DEAD tasks
ELF loader	+3 days	Parse ELF, load, test “hello”
Userland infrastructure	+2 days	C runtime (crt0), basic libc calls
Shell & utilities	+4 days	Build `/bin`, implement `execv`, I/O syscalls
File system (FAT/VFS)	+1 week	Implement block devices, simple fs
Advanced (SMP/UEFI)	much later	(beyond initial scope)

(These dates are illustrative; actual schedule may vary.)

Summary: The kernel is solid at the current point – you’ve effectively built a toy 32-bit UNIX-like kernel. The **next key goal** is a complete **user-space environment**: loader, C library, shell, etc. Before that, fix the few critical bugs and polish memory isolation. Once an `/bin/hello` returns properly, you’ve truly “gotten to userspace.”

Sources: Concepts here are supported by standard OS references and OSDev guides【50†L17-L22】【53†L344-L349】【57†L18-L27】【66†L171-L179】【59†L21-L24】, with code snippets inferred from the current `osdev` codebase structure.
